



Handwritten Digit Recognition using Convolutional Neural Network (CNN)

Recognizing Digits with Advanced Neural Networks

Deep Learning Project Report

Title

Handwritten Digit Recognition using Convolutional Neural Network (CNN)

1. Introduction

This project aims to classify handwritten digits (0–9) using a Convolutional Neural Network (CNN). Handwritten digit recognition is one of the most common image classification problems in Deep Learning. CNN models are highly effective for image processing tasks because they automatically extract important visual features from images. The goal of this project is to train a CNN model capable of recognizing handwritten digits with high accuracy using the MNIST dataset.

2. Dataset

We used the MNIST dataset, which contains grayscale images of handwritten digits.

- **Image size:** 28×28 pixels
- **Number of classes:** 10 (digits from 0 to 9)
- **Total images:** 70,000

- **Training images:** 60,000
- **Testing images:** 10,000

The dataset is divided into training and testing sets to evaluate the model's performance.

3. Data Preprocessing

The following preprocessing steps were applied:

- Converted images to tensors
- Normalized pixel values to range [0,1]
- Reshaped images into $(28 \times 28 \times 1)$
- Applied one-hot encoding to labels
- Split the dataset into training and testing sets

These preprocessing steps help improve the performance and training speed of the CNN model.

4. Model Architecture (CNN)

The CNN model consists of the following layers:

1. **Convolutional Layer** (32 filters, 3×3 kernel, ReLU activation)
2. **Max Pooling Layer** (2×2)
3. **Convolutional Layer** (64 filters, 3×3 kernel, ReLU activation)
4. **Max Pooling Layer** (2×2)
5. **Flatten Layer**
6. **Fully Connected Dense Layer** (128 neurons, ReLU activation)
7. **Dropout Layer** (0.5)
8. **Output Layer** (10 neurons, Softmax activation)

The CNN architecture is designed to extract image features efficiently and improve classification accuracy.

5. Model Enhancement

The following techniques were used to improve model performance:

- **Dropout** to reduce overfitting
- **ReLU activation function**
- **Adam optimizer** for faster convergence

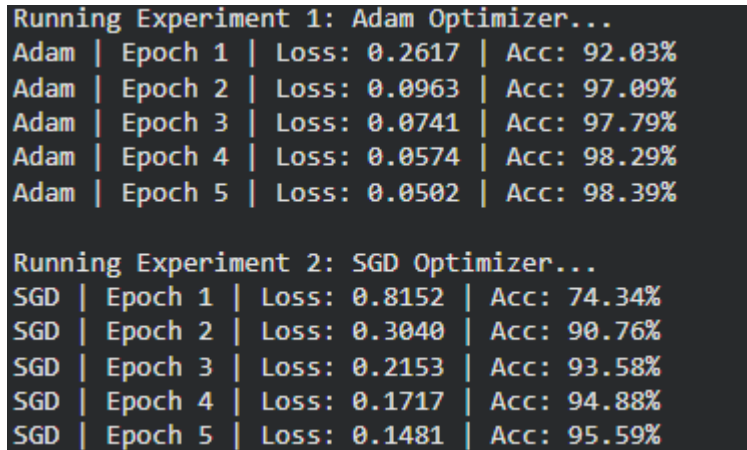
These techniques improved the stability and accuracy of the model during training.

6. Training

The model was trained using the following parameters:

- **Loss Function:** CrossEntropyLoss
- **Optimizer:** Adam
- **Number of epochs:** 5
- **Batch size:** 64

The training process was monitored using accuracy and loss metrics.



```
Running Experiment 1: Adam Optimizer...
Adam | Epoch 1 | Loss: 0.2617 | Acc: 92.03%
Adam | Epoch 2 | Loss: 0.0963 | Acc: 97.09%
Adam | Epoch 3 | Loss: 0.0741 | Acc: 97.79%
Adam | Epoch 4 | Loss: 0.0574 | Acc: 98.29%
Adam | Epoch 5 | Loss: 0.0502 | Acc: 98.39%

Running Experiment 2: SGD Optimizer...
SGD | Epoch 1 | Loss: 0.8152 | Acc: 74.34%
SGD | Epoch 2 | Loss: 0.3040 | Acc: 90.76%
SGD | Epoch 3 | Loss: 0.2153 | Acc: 93.58%
SGD | Epoch 4 | Loss: 0.1717 | Acc: 94.88%
SGD | Epoch 5 | Loss: 0.1481 | Acc: 95.59%
```

Screenshot of the training process execution logs, showing the loss reduction and accuracy improvement across the 5 epochs for both Adam and SGD optimizers

7. Experiments

Two experiments were conducted to compare the performance of different optimizers.

Experiment 1

- **Optimizer:** Adam
- **Accuracy:** 98.39%
- **Loss:** 0.0502

Experiment 2

- **Optimizer:** SGD
- **Accuracy:** 95.59%
- **Loss:** 0.1481

The experiments were performed to evaluate how different optimizers affect model performance.

8. Results

Final Accuracy

- **Experiment 1 (Adam):** 98.39%
- **Experiment 2 (SGD):** 95.59%

Comparison Table

Model	Optimizer	Accuracy	Loss
CNN Model A	Adam	98.39%	0.0502
CNN Model B	SGD	95.59%	0.1481

```
Running Experiment 1: Adam Optimizer...
Adam | Epoch 1 | Loss: 0.2617 | Acc: 92.03%
Adam | Epoch 2 | Loss: 0.0963 | Acc: 97.09%
Adam | Epoch 3 | Loss: 0.0741 | Acc: 97.79%
Adam | Epoch 4 | Loss: 0.0574 | Acc: 98.29%
Adam | Epoch 5 | Loss: 0.0502 | Acc: 98.39%

Running Experiment 2: SGD Optimizer...
SGD | Epoch 1 | Loss: 0.8152 | Acc: 74.34%
SGD | Epoch 2 | Loss: 0.3040 | Acc: 90.76%
SGD | Epoch 3 | Loss: 0.2153 | Acc: 93.58%
SGD | Epoch 4 | Loss: 0.1717 | Acc: 94.88%
SGD | Epoch 5 | Loss: 0.1481 | Acc: 95.59%
```

Screenshot of the training process execution logs, showing the loss reduction and accuracy improvement across the 5 epochs for both Adam and SGD optimizers

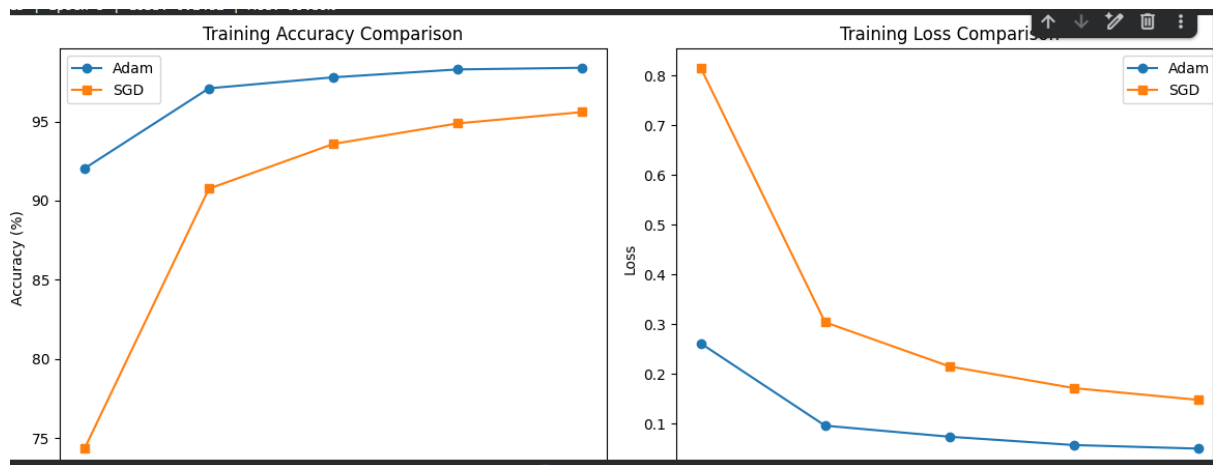
9. Discussion

The results show that the CNN model achieved very high accuracy on the MNIST dataset. The Adam optimizer provided better performance and faster convergence compared to SGD. The use of convolutional layers helped the model extract important image features effectively. Dropout also helped reduce overfitting and improved generalization on test data.

10. Visualization

The following graphs are included in the project:

- Training Accuracy Comparison Curve (Adam,SGD)
- Validation Accuracy Curve
- Training Loss Comparison Curve (Adam,SGD)
- Validation Loss Curve



Comparison curves between Adam and SGD optimizers. The left graph shows that Adam achieves higher training accuracy faster, while the right graph shows that Adam minimizes the training loss significantly better than SGD over the 5 epochs.

11. Conclusion

This project demonstrated the effectiveness of Convolutional Neural Networks (CNNs) in handwritten digit recognition tasks. The model achieved high classification accuracy on the MNIST dataset and showed strong performance in image classification problems. Experimental comparisons showed that the Adam optimizer achieved better results than SGD.

12. Future Improvements

The following improvements can be added in future work:

- **Batch Normalization**
- **Data Augmentation**
- **More CNN layers**
- **Hyperparameter tuning**
- **Testing on larger datasets**